

Water Pump Fault Detection

using TinyML and Edge Impulse

Arduino Nano 33 BLE Sense • ACS712 • ADS1115 • SH1106 OLED

Project Type	TinyML / Predictive Maintenance
Board	Arduino Nano 33 BLE Sense
Platform	Edge Impulse Studio
Submission	Office / Academic Submission
Status	Final Report

1. Introduction

This project implements a TinyML-based Water Pump Fault Detection System using the Arduino Nano 33 BLE Sense board and Edge Impulse. The system performs real-time monitoring and classification of pump operating conditions using onboard sensors such as the accelerometer (IMU) and current sensing hardware.

The trained Edge Impulse model is deployed on the Arduino Nano 33 BLE Sense board, where the system performs live inference and indicates detected conditions using LEDs, OLED display, and buzzer.

2. Objectives

The main objectives of this project are:

- To collect vibration and current data from a water pump.
- To train a TinyML model using Edge Impulse.
- To deploy the trained model on Arduino Nano 33 BLE Sense.
- To classify different pump conditions in real time.
- To provide visual and audible fault indication.

3. Components Used

Component	Qty	Purpose
Arduino Nano 33 BLE Sense	1	Main TinyML microcontroller board
ACS712 Current Sensor	1	Current sensing (pump load)
ADS1115 ADC Module	1	High-resolution 16-bit ADC
SH1106 OLED Display	1	Real-time status display (128×64)
LEDs (5×)	5	Fault condition indication
220Ω Resistors (5×)	5	Current-limiting for LEDs
Buzzer	1	Audible fault alert
Breadboard	1	Prototyping platform
DC Submersible Water Pump	1	Device under test
Jumper Wires	Multiple	Component connections
5V / 12V Power Supply	1	Pump & board power

4. Software and Tools Used

Software / Tool	Purpose
Arduino IDE	Firmware development and flashing
Edge Impulse Studio	Dataset management, training & deployment
Edge Impulse CLI	Serial data acquisition from board
Python / CSV	Initial dataset experimentation
macOS Terminal	CLI tools and board management

5. Block Diagram

The diagram below shows all hardware connections between the Arduino Nano 33 BLE Sense, OLED display, ADS1115 ADC, ACS712 current sensor, LEDs, buzzer, and water pump.

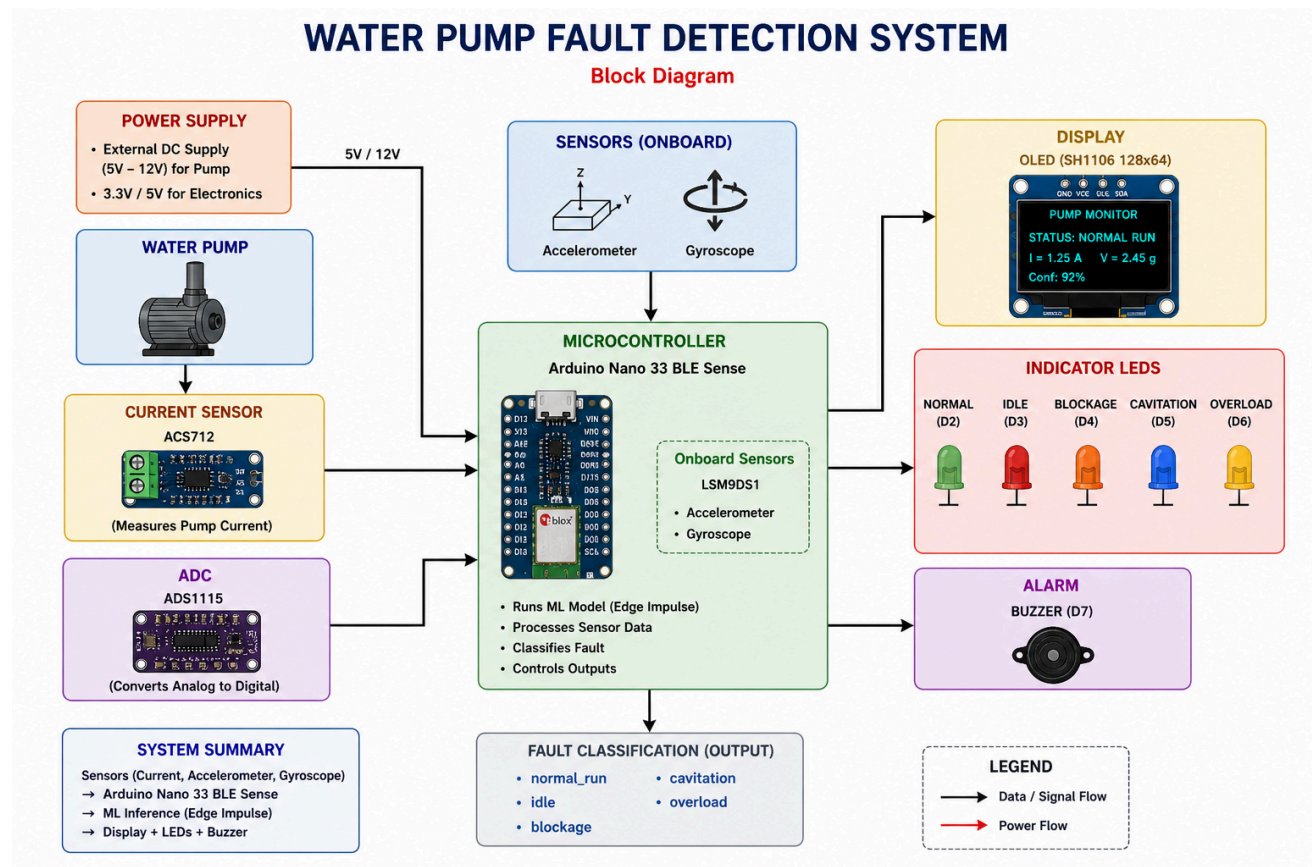


Figure 1 — Complete Circuit Schematic: Water Pump Fault Detection System

5.1 OLED (SH1106) Connections

OLED Pin	Arduino Nano 33 BLE Sense Pin	Wire Colour
VCC	3.3V	Red
GND	GND	Black
SDA	A4 (SDA)	Yellow
SCL	A5 (SCL)	Orange

5.2 LED Connections (with 220Ω Resistors)

Fault Condition	Arduino Pin	LED Colour
normal_run	D2	Green
idle	D3	Blue
blockage	D4	Orange
cavitation	D5	Purple
overload	D6	Red

5.3 Buzzer Connection

Buzzer Terminal	Arduino Pin
+ (Positive)	D7
- (Negative)	GND

5.4 ACS712 Current Sensor Connections

ACS712 Signal Connections

ACS712 Pin	Connection
VCC	5V from Arduino
GND	GND
OUT	ADS1115 A0 Input

ACS712 Power Path (Pump Circuit)

Connection	Description
Power Supply (+) → ACS712 IP+	Pump positive supply input
ACS712 IP- → Pump (+)	Measured current output to pump
Pump (-) → Power Supply GND	Ground return path

5.5 ADS1115 ADC Connections

ADS1115 Pin	Arduino Nano 33 BLE Sense
VCC	3.3V
GND	GND
SDA	A4 (I2C Data)
SCL	A5 (I2C Clock)
A0	ACS712 OUT (Analog Input)

6. Initial Dataset and Learning Phase

Initially, a public Kaggle dataset related to pump sensor data was used for experimentation and understanding the Edge Impulse workflow.

6.1 Dataset Contents

- Pump operating conditions across multiple states
- Multiple sensor values per record
- CSV-based training data

6.2 Classification Labels

Label	Description
normal_run	Pump operating under normal load
idle	Pump powered but not pumping
blockage	Outlet/inlet blockage detected
cavitation	Air cavity formation in pump
overload	Excessive current / mechanical stress

6.3 Learning Outcomes

- Understanding TinyML data pipeline
- Training and deployment process on Edge Impulse
- Feature generation and windowing
- Confusion matrix analysis and model optimization

Note: The Kaggle dataset did not perfectly match the actual hardware setup. Real hardware data collection was subsequently performed.

7. Feature Engineering

7.1 Initial Feature Set

The following six features were extracted from sensor readings:

Feature	Source	Description
ax	IMU Accelerometer	X-axis acceleration
ay	IMU Accelerometer	Y-axis acceleration
az	IMU Accelerometer	Z-axis acceleration
current	ACS712 via ADS1115	Raw pump current
accel_rms	Computed	RMS of acceleration vector
abs(current)	Computed	Absolute current value

8. TinyML Workflow

Step 1 — Data Collection

- Edge Impulse Data Forwarder (serial streaming)
- CSV dataset upload
- Sensor streaming through serial monitor
- Hear data is collected using the vibration of bread board
- But in real thai cases are different (Here it is taken just to test the model is working or not0

Fault Case	Vibration
normal_run	Pump on with normal vibration with hum sound
idle	No sound and pump off
blockage	More amount of vibration
overload	Moderate vibration
cavitation	Rotation

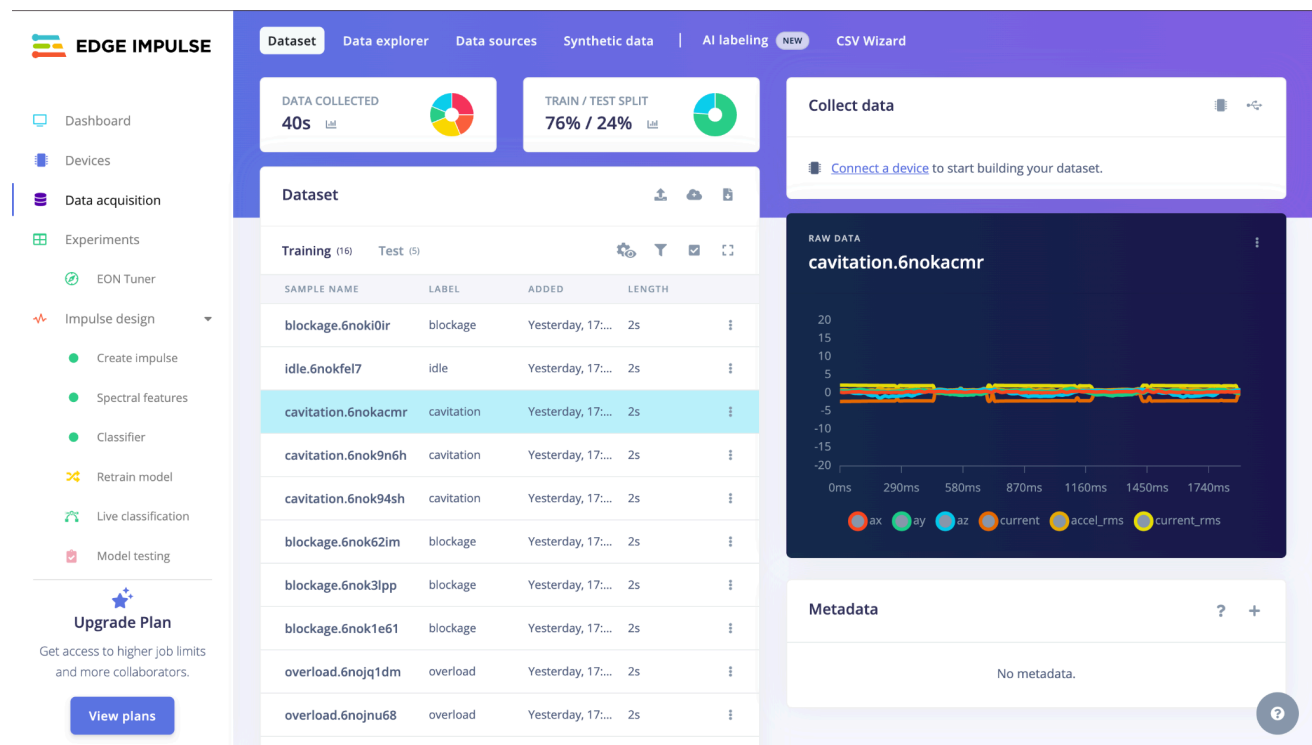


Figure 2 — Data Collection on Edge Impulse

Step 2 — Feature Generation (Edge Impulse)

Parameter	Value
Feature Extractor	Spectral Analysis
Sampling Rate	100 Hz
Window Size	2000 ms
Window Stride	500 ms

Step 3 — Model Training

- Classifier: Neural Network (fully connected)
- Labels: 5 fault classes
- Train / Test split: 80% / 20%

Step 4 — Deployment

The trained model was exported as an Arduino Library (.zip) and deployed using Arduino IDE.

Here the Library name is `WaterPumpFaultDetectionCustom_inferencing.h`

9. Final Deployed Firmware

9.1 Libraries Used

Library	Purpose
Arduino_LSM9DS1	IMU accelerometer + gyroscope
Adafruit_ADS1X15	ADS1115 16-bit ADC driver
U8g2	SH1106 OLED display driver
Edge Impulse Inferencing	On-device TinyML inference
Wire	I2C communication bus

9.2 Real-Time Inference Loop

1. Read accelerometer data (ax, ay, az) from IMU
2. Read current via ADS1115 ← ACS712 sensor
3. Compute accel_rms and abs(current)
4. Build feature buffer for inference window
5. Run TinyML inference (Edge Impulse model)
6. Parse prediction label and confidence score
7. Display result on SH1106 OLED
8. Activate corresponding LED (D2–D6)
9. Trigger buzzer for non-normal conditions

9.3 Code

```
// EDGE IMPULSE + REAL SENSOR MODEL (FINAL VERSION)

// =====

#include <WaterPumpFaultDetectionCustom_inferencing.h>

#include <Arduino_LSM9DS1.h>

#include <Wire.h>

#include <Adafruit_ADS1X15.h>

#include <U8g2lib.h>

// OLED

U8G2_SH1106_128X64_NONAME_F_HW_I2C u8g2(U8G2_R0, U8X8_PIN_NONE);
```

```
// ADS1115

Adafruit_ADS1115 ads;

// LED pins

#define LED_NORMAL    2

#define LED_IDLE      3

#define LED_BLOCKAGE  4

#define LED_CAVIT     5

#define LED_OVERLOAD  6

#define BUZZER_PIN    7


const int ALL_LEDS[5] = {2,3,4,5,6};


// Labels (must match Edge Impulse)

const char* LABELS[5] = {

    "normal_run",

    "idle",

    "blockage",

    "cavitation",

    "overload"

};


const int CLASS_LED[5] = {

    LED_NORMAL,

    LED_IDLE,
```

```
LED_BLOCKAGE,  
  
LED_CAVIT,  
  
LED_OVERLOAD  
};  
  
// Feature buffer  
  
float features[EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE];  
  
int fi = 0;  
  
// Global variables  
  
float g_current = 0;  
  
float g_accel_rms = 0;  
  
// ACS offset  
  
float offset = 2.5;  
  
// =====  
  
// CALIBRATE ACS712  
  
// =====  
  
void calibrateACS() {  
  
    float sum = 0;  
  
    Serial.println("Calibrating current sensor... keep pump OFF");  
  
    for (int i = 0; i < 200; i++) {  
  
        float v = ads.computeVolts(ads.readADC_SingleEnded(0));
```

```
    sum += v;

    delay(5);

}

offset = sum / 200.0;

Serial.print("Offset: ");

Serial.println(offset);

}

// =====

// READ CURRENT

// =====

float readCurrent() {

    float v = ads.computeVolts(ads.readADC_SingleEnded(0));

    return (v - offset) / 0.185;

}

// =====

void setLED(int cls) {

    for (int i = 0; i < 5; i++) digitalWrite(ALL_LEDS[i], LOW);

    digitalWrite(CLASS_LED[cls], HIGH);

}

// =====

void buzzIfFault(int cls) {
```

```
if (cls == 0 || cls == 1) noTone(BUZZER_PIN); // normal or idle

else tone(BUZZER_PIN, 2000, 200);

}

// =====

void drawOLED(int cls, float conf, float current, float accel) {

    u8g2.clearBuffer();

    u8g2.setFont(u8g2_font_6x10_tf);

    u8g2.drawStr(0, 10, "PUMP MONITOR");

    u8g2.setFont(u8g2_font_8x13B_tf);

    u8g2.drawStr(0, 28, LABELS[cls]);

    char buf[32];

    sprintf(buf, "Conf:%d%%", (int)(conf * 100));

    u8g2.drawStr(0, 45, buf);

    sprintf(buf, "I=%.2fA", current);

    u8g2.drawStr(0, 60, buf);

    sprintf(buf, "V=%.2fg", accel);

    u8g2.drawStr(70, 60, buf);

    u8g2.sendBuffer();
```

```
}

// =====

int feature_get_data(size_t offset, size_t length, float *out_ptr) {

    memcpy(out_ptr, features + offset, length * sizeof(float));

    return 0;

}

// =====

void setup() {

    Serial.begin(115200);

    // LEDs

    for (int i = 0; i < 5; i++) pinMode(ALL_LEDS[i], OUTPUT);

    pinMode(BUZZER_PIN, OUTPUT);

    // OLED

    u8g2.begin();

    // IMU

    if (!IMU.begin()) {

        Serial.println("IMU FAIL");

        while (1);

    }

}
```

```
// ADC

ads.begin();

ads.setGain(GAIN_ONE);

calibrateACS();

Serial.println("System Ready");
}

// =====

void loop() {

    fi = 0;

    float accel_sum = 0;

    // ===== Collect features =====

    while (fi < EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE - 6) {

        float ax, ay, az;

        if (IMU.accelerationAvailable()) {

            IMU.readAcceleration(ax, ay, az);

            float current = readCurrent();

            float accel_rms = sqrt(ax*ax + ay*ay + az*az);
```

```
// 6 features (MUST MATCH TRAINING)

features[fi++] = ax;

features[fi++] = ay;

features[fi++] = az;

features[fi++] = current;

features[fi++] = accel_rms;

features[fi++] = fabs(current);

accel_sum += accel_rms;

}

delay(10); // 100 Hz

}

g_current = readCurrent();

g_accel_rms = accel_sum / (fi / 6);

// ===== Run classifier =====

signal_t signal;

signal.total_length = EI_CLASSIFIER_DSP_INPUT_FRAME_SIZE;

signal.get_data = &feature_get_data;

ei_impulse_result_t result;

run_classifier(&signal, &result, false);
```

```
int best = 0;

float best_val = 0;

for (int i = 0; i < EI_CLASSIFIER_LABEL_COUNT; i++) {

    if (result.classification[i].value > best_val) {

        best_val = result.classification[i].value;

        best = i;

    }

}

// ===== Output =====

setLED(best);

buzzIfFault(best);

drawOLED(best, best_val, g_current, g_accel_rms);

Serial.print("Class: ");

Serial.print(LABELS[best]);

Serial.print(" | Current: ");

Serial.println(g_current);

delay(200);

}
```

10. Experimental Observations

10.1 Circuit Implementation on the Bread-board

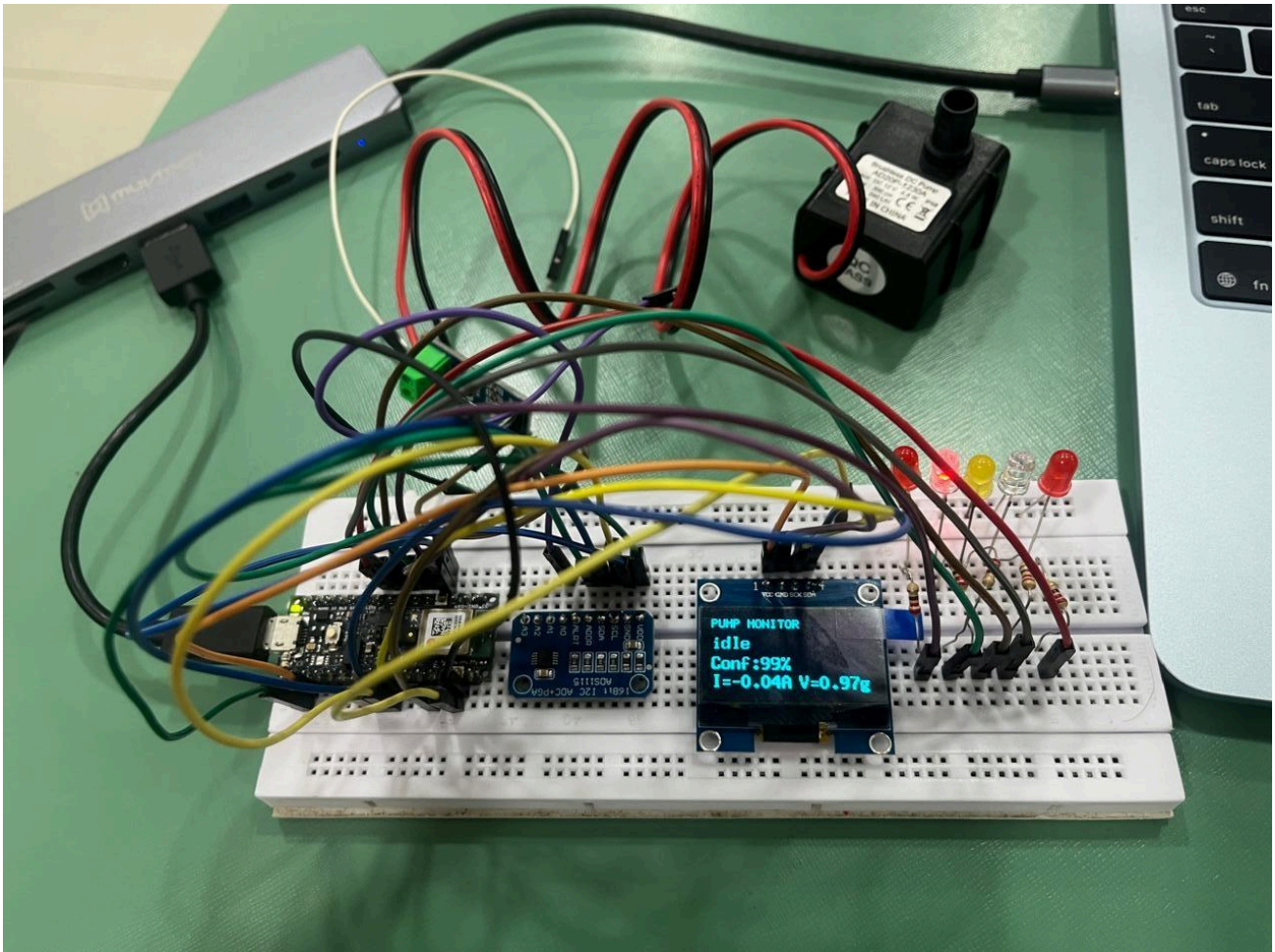


Figure 3 — Complete Circuit Implementation of Water Pump Fault Detection System On Bread-Board

10.2 Current Sensor Limitations

The ACS712 sensor showed very small current variation during testing, reducing class separability.

Root causes identified:

- Pump operating current was very low — below ACS712's sensitive range
- Sensor noise floor masked actual current variations
- ACS712 is designed for higher current ranges (5A / 20A / 30A variants)

Resulting issues:

- Similar feature values across different fault conditions
- Weak class separation in feature space
- Occasional false overload detection

10.3 Vibration Coupling Issue

The Nano 33 BLE Sense was mounted on a breadboard rather than directly on the pump body. Manual vibration was applied to the breadboard during data collection, resulting in:

- Weak real-world vibration signatures
- Poor sensor-to-pump coupling
- Insufficient differentiation between fault conditions

11. Key Learning

If sensor data is weak or inconsistent:

- Model accuracy decreases regardless of network architecture
- Different fault classes overlap in feature space
- Fault detection becomes unreliable in deployment

12. Areas of Improvement

12.1 Replace ACS712 with INA219

Aspect	ACS712 (Current)	INA219 (Recommended)
Interface	Analog output	Digital I2C
Accuracy	Low for small A	High precision
Noise	High	Low
Measurement	Current only	Voltage + Current + Power
Calibration	Manual offset	Built-in calibration register

12.2 Use Onboard Microphone

The Nano 33 BLE Sense includes a PDM microphone for acoustic analysis:

- Cavitation sound detection (high-frequency signatures)
- Mechanical vibration noise analysis
- Recommended features: Audio RMS, Spectrogram, MFE (Mel Filter Energy)

12.3 Mount Board Directly on Pump

- Use double-sided tape, zip ties, or 3D-printed brackets
- Improves vibration transfer to IMU significantly
- Increases IMU sensitivity and fault separability

12.4 Add Gyroscope Features

Recommended expanded feature set:

Feature	Sensor	Captures
ax, ay, az	LSM9DS1 Accelerometer	Linear vibration
gx, gy, gz	LSM9DS1 Gyroscope	Rotational instability

12.5 Use Official Edge Impulse Firmware

- Stable, timestamped sensor sampling
- Native IMU and microphone support
- Automatic data labelling and upload
- Eliminates custom serial firmware complexity

13. Recommended Future Architecture

Sensor	Interface	Purpose
LSM9DS1 IMU	SPI (onboard)	Linear + rotational vibration
PDM Microphone	PDM (onboard)	Acoustic fault signatures
INA219	I2C	Current, voltage & power
Flow Sensor	Digital pulse	Water flow rate monitoring

This multi-sensor fusion approach would deliver a significantly more reliable and industrial-grade predictive maintenance system.

14. Conclusion

This project successfully demonstrated:

- TinyML model deployment on resource-constrained embedded hardware
- Real-time fault classification using Edge Impulse
- Sensor-based predictive maintenance concepts
- OLED + LED + Buzzer based multi-modal fault indication

Practical understanding was gained in:

- Feature engineering and sensor selection
- Real-world signal quality challenges
- Hardware limitations and workarounds
- Embedded AI deployment workflow

Despite limitations from breadboard mounting and ACS712 sensor sensitivity, the project establishes a strong and reproducible foundation for an industrial-grade predictive maintenance system.

15. Future Scope

Enhancement	Description
Cloud Connectivity	MQTT / HTTP telemetry to cloud dashboard
IoT Dashboard	Real-time monitoring via web/mobile app
Auto Shutdown	GPIO relay to cut power on critical fault
Anomaly Detection	Unsupervised learning for unknown faults
Industrial Testing	Trials on larger industrial pumps
Multi-sensor Fusion	Combine IMU + mic + current + flow
Edge AI Optimization	INT8 quantization, model pruning
Mobile Alerts	Push notifications via BLE / Wi-Fi